1. Suppose we are able to do MiniMax search over the entire search tree. The strategy developed using the values backed up from this search will be optimal against every player?
    a. True
    b. False

    MiniMax over the entire search tree will yield optimal strategy only against an optimal player, but if the opponent is optimal then MiniMax will not be able to exploit the weakness of the opponent.

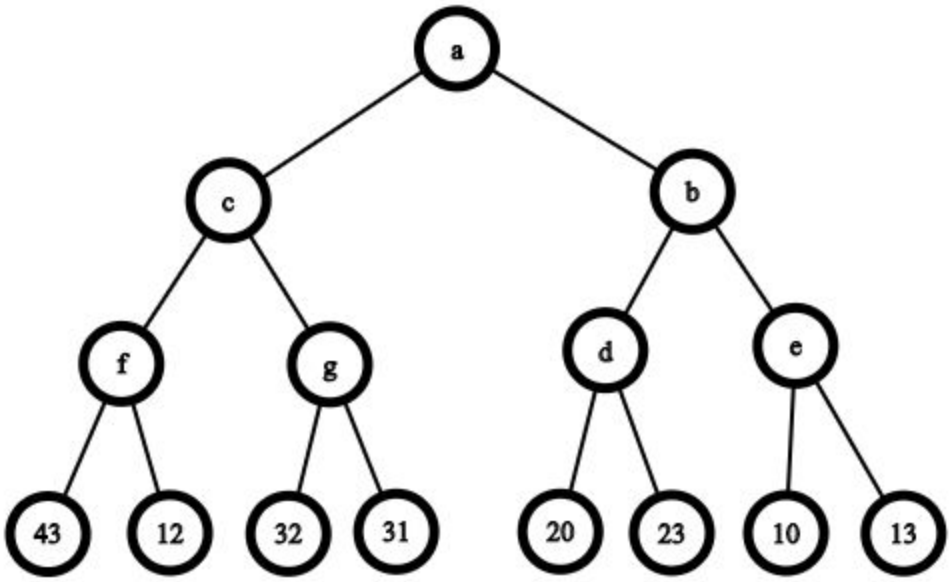
2. You know from experience that the value of a particular state in a MiniMax search tree lies between x and y (x < y). How can you use this information to optimize your alpha-beta search?
    a. Set α = x and β = y
    b. Set α = y and β = x
    c. Return the value of state as (x+y)/2
    d. This information cannot be used to optimize the procedure

    Alpha and beta are the lower and upper bound respectively while searching. If we know that a node cannot have a value less than x and greater than y then we can tighten our bounds and thus prune possibly more nodes.

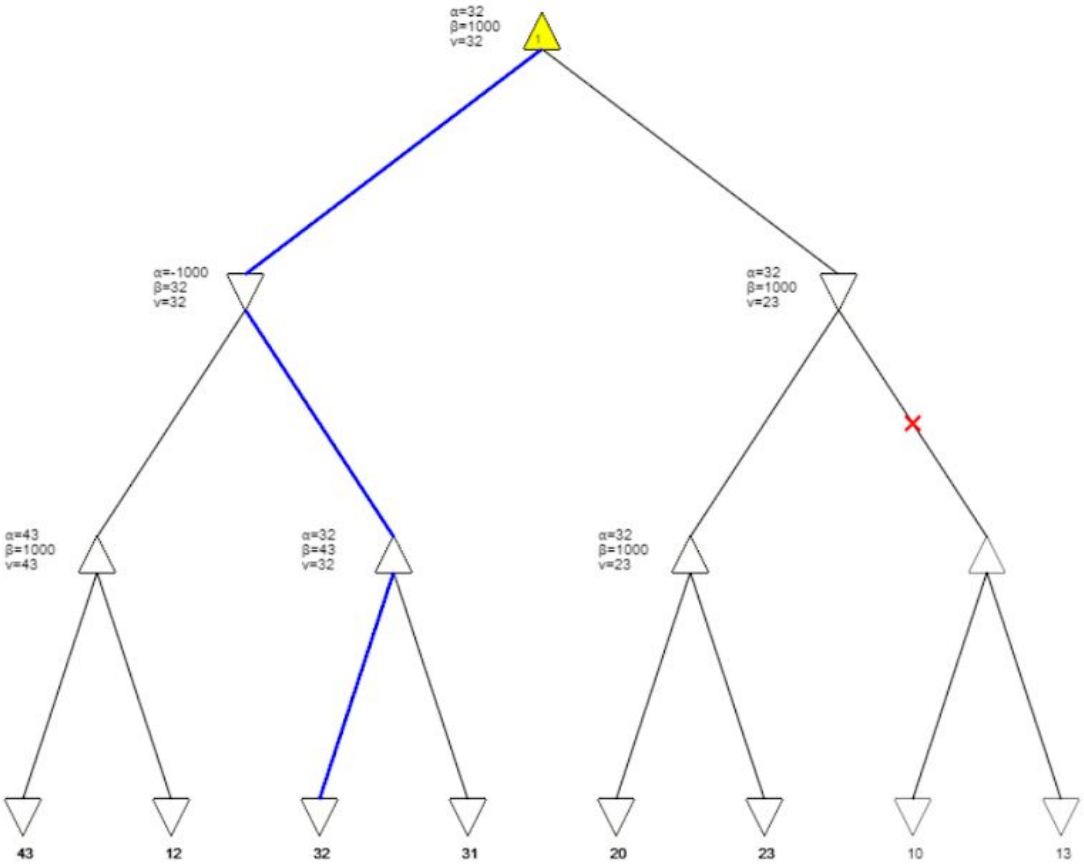
3. Suppose you are doing MiniMax search with alpha-beta pruning to solve the game of chess. You are at a Max Node and plan to do 6-ply lookahead from here. Which of the following strategies can help you in reducing the number of nodes explored?
    a. Doing a 2-ply lookahead and then reordering the successor Min nodes in descending order of their values
    b. Doing a 2-ply lookahead and then reordering the successor Min nodes in ascending order of their values
    c. Using domain knowledge (if there exists any) to set tighter alpha-beta bounds

    Reordering of nodes in descending order while searching from max nodes can improve the effective branching factor by doing more pruning (See lecture slides for this). Similarly setting tighter bounds helps in rejecting nodes quickly and improves efficiency.

4. What is the value of the root node after MiniMax search over the given tree? The root node (labeled ‘a’) is a max node.

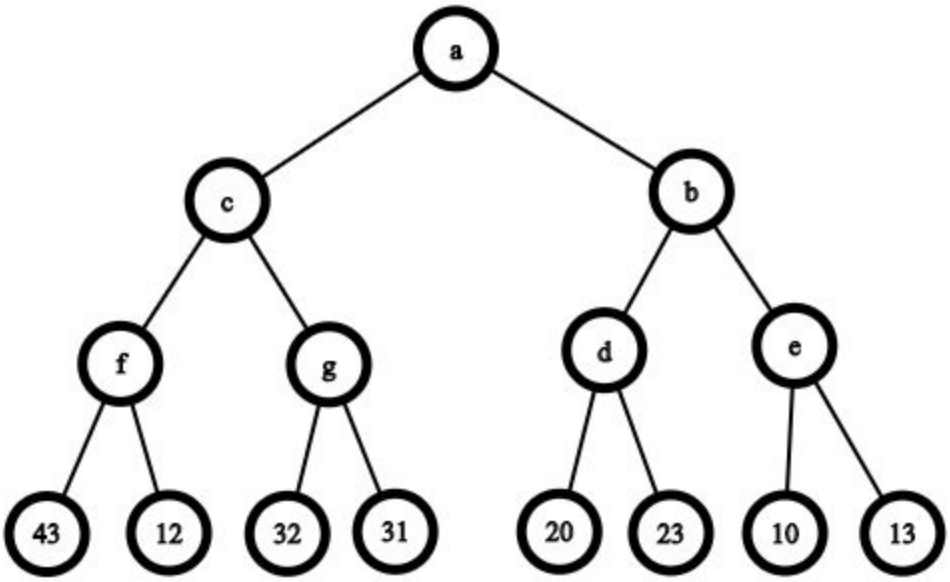


Ans. **32**



The number on root here is the node id (not the value of root). The value of each node is indicated by v.

5. How many nodes (including leaves and internal nodes) will an alpha-beta pruning based MiniMax search visit in the given tree, assuming each successor is ordered left to right? The root node (labeled ‘a’) is a max node.



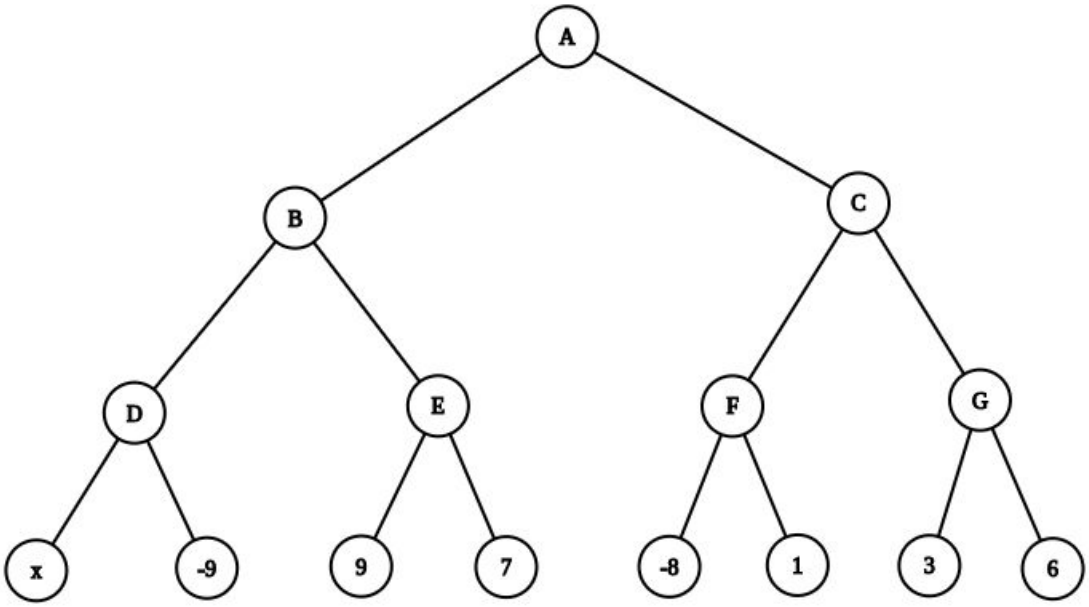
Ans. **12**

See the image for Q4.

6. Which of the following is true about Alpha-Beta pruning?
    a. It may not find the optimal play against an optimal opponent.
    b. The no. of pruned nodes is independent of the order of exploration.
    c. In the worst case, it may not prune any nodes.
    d. None of the above

The assumption of the minimax algorithm is that the opponent is optimal so (a) is incorrect. The order of exploration strongly affects the number of pruned nodes if good moves or bad moves are encountered first depending on whether it is a max or min node so (b) is wrong. In the worst scenario, we may have to look at every node and no pruning may occur, therefore (c) is correct.

7. Player 1 (max node) chooses the left action for which of the following value(s) of x. Assume optimal play and Player 1's turn at Node A.



   a. 2
   b. -7
   c. 4
   d. -1

   Let the value at a node X be denoted by V(X). Then, V(D) = max(x, -9), V(E) = 9, V(F) = 1, V(G) = 6. Therefore, V(B) = min(max(x, -9), 9) and V(C) = 1.
   We need V(B) > V(C) i.e. min(max(x, -9), 9) > 1.
   min(a, b) > c when a > c and b > c.
   Therefore, we want max(x, -9) > 1. (9 > 1 is true)
   max(a, b) > c when a > c or b > c.
   Therefore, we want x > 1. (-9 > 1 is false)
   Hence, options (a) and (c) are correct.

8. There exists a perfect evaluation function at each node, such that no search is needed in the minimax algorithm.
   a. True
   b. False

   A function that assigns the values to every node so that it makes the player choose the action that a full tree search would have found is such a function. Note that such a function exists but it is rarely known. Also, such a function may not have a closed-form in

terms of features of the state. It might in fact be a full mapping from every state to a value.

9. In the general minimax search paradigm, the evaluation function used to calculate the utility of a min node is negative of the evaluation function used to calculate the utility of a max node.
    a. True
    b. False

    The evaluation function remains the sum. We choose the min or max function on the values depending on the depth

10. If the set of successors are ordered randomly then the complexity of minimax with alpha-beta pruning (for moderate b) is closest to
    a. $O(b^{3m/4})$
    b. $O(b^{m/2})$
    c. $O(b^{m})$
    d. $O(b^{2m/})$

    The number of nodes will be roughly $O(b^{3m/4})$. The calculation is not trivial. One might intuitively think as some geometric mean of the best and worst-case possibilities.